A Categorical Framework for Visualization Software Architecture

Hannah Aizenman *
Computer Science, The Graduate Center, CUNY
Mikael Vejdemo-Johansson †
Mathematics, CUNY College of Staten Island
Thomas Caswell ‡
National Synchrotron Light Source II, Brookhaven National Lab
Michael Grossberg §
Computer Science, City College of New York, CUNY

Abstract

????

1 Intro

Graphical grammars are used to express how to put various visual elements together; we propose a methodology for fully specifying arbitrary visual elements in a manner that translates to pseudocode. We propose that fiber bundles provide a uniform interface for describing a variety of underlying data - tables, images, networks, etc. - in a manner that independently encodes the mathematical structure of the topology and the fields of the dataset. Modeling the data structures that store the datasets as sheaves provides a framework for specifying visualization methods that are designed to work regardless of how the dataset is stored - whether the data is on disk, distributed, or on demand. Specifying the visualization library components as natural transforms of sheaves means that the constraints that the component must satisfy to be structure preserving can be specified as the set of morphisms on the data and graphic sheaves, including the structure on the continuity and fields of the data. Using category theory to formally express how visual elements are constructed means we can translate those constraints into pseudocode, which can be used to implement the words in the visual sentence in way that ensures they're structure preserving.

2 Related Work

3 Uniform Abstraction for Data & Graphics

3.0.1 Example: Uniform Abstract Graphic Representation

3.0.2 Data transformations

3.1 Abstract Data Containers

axiom	applied to datasets and indexes
presheaf	given $\text{index}_1 \subset \text{index}_2$: $\exists\ \text{dataset}[\text{index}_2]$ $\Rightarrow \text{dataset}[\text{index}_2][\text{index}_1] = \text{dataset}[\text{index}_1]$ $\exists\ \text{dataset}[\text{index}_1] \not\Rightarrow \exists\ \text{dataset}[\text{index}_2]$
locality	$\text{dataset}^1[i] = \text{dataset}^2[i]\ \forall\ i \in \text{index}$ $\Rightarrow \text{dataset}^1 = \text{dataset}^2$
gluing	$i = j$ and $\text{dataset}^1[i] = \text{dataset}^2[j]$ $\text{dataset}^3 \coloneqq \text{dataset}^1[:i] \oplus \text{dataset}^2[j:]$

Table 1: When these axioms are satisfied, we can treat the data container as a sheaf, which means local data availability and consistency constraints are modeled by inclusion maps $\hat{\phi}$ and the resulting restriction maps $\hat{\phi}^*$.

*email:haizenman@gradcenter.cuny.edu
†e-mail: mvj@math.csi.cuny.edu
‡e-mail: tcaswell@bnl.gov
§e-mail: mgrossberg@ccny.cuny.edu

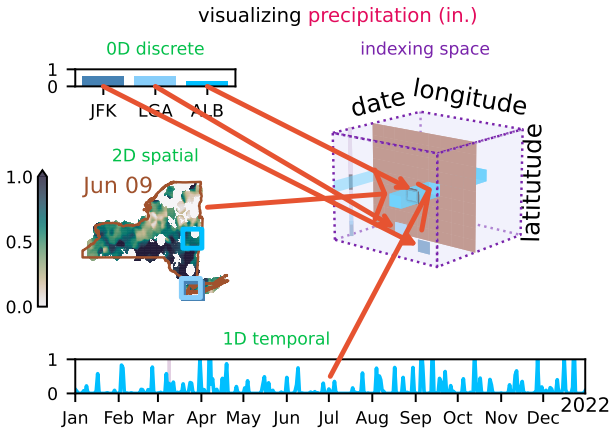


Figure 1: This weather station data has multiple embedded continuities - points at each time and position, timeseries at each position, and maps at each time. The corresponding visualizations - bar chart, timeseries, and map - each preserve the continuity of the subset of the data they visualize by not introducing or leaving out values and preserving the relative positioning of continuous values.

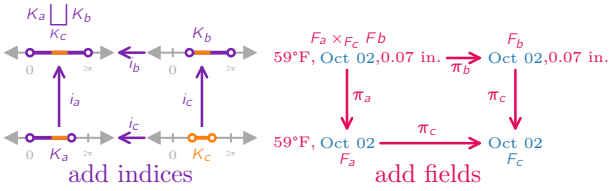


Figure 2: Records are added to the dataset as the *fibered coproduct* $K_a \sqcup_{K_c} K_b$ of index spaces. Fields are added to each record as the *fiber product* $F_a \times_{F_c} F_b$ of fiber spaces.

4 Connectivity Preserving Map: Readability ξ

5 Artist: Structure Preserving Maps

5.1 Composing Artists

6 Constructing Structure Preserving Components

6.1 Verification

7 Conclusion

Supplemental Materials

This paper is based on the theoretical work described in `https://github.com/story645/team/paper/`.

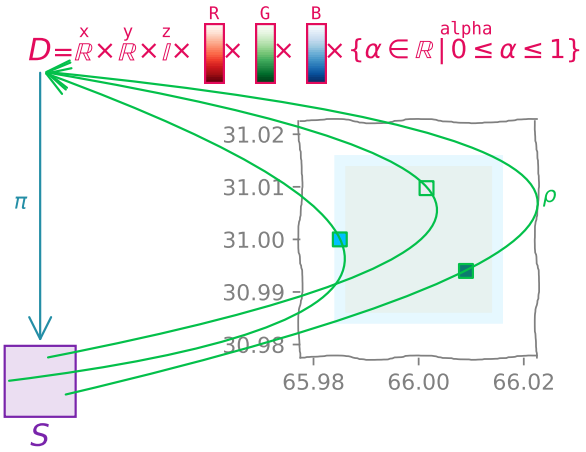


Figure 3: The pixels in the scatter marker are the records returned by the section function ρ over the points s ∈ S.

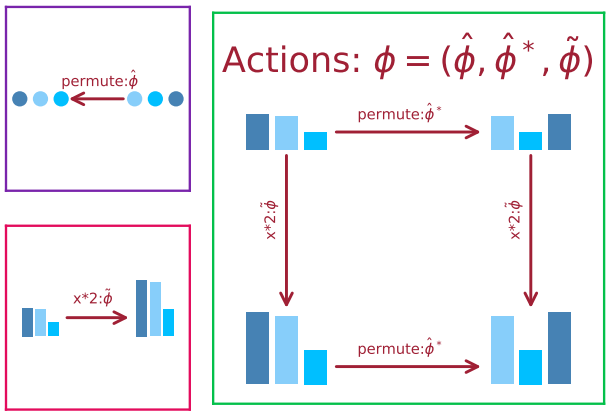


Figure 4: The actions on the base and fiber space commute such that the data can be permuted then scaled or scaled then permuted.

ACKNOWLEDGEMENTS

The authors would like to thank the anonymous reviewers who gave constructive feedback on an earlier version of this paper. The authors are also grateful to the various Matplotlib and Napari contributors, particularly Juan Nunez-Iglesias, and Nicolas Kruchten for their valuable feedback. Hannah is also very grateful to Nicolas for the suggestion of augmented notation and to the nlab and wikipedia contributors who wrote clear explanations of many of the topics discussed in this paper. This project has been made possible in part by grant number 2019-207333 (EOSS Cycle 1) and EOSS Cycle 3 from the Chan Zuckerberg Initiative DAF, an advised fund of Silicon Valley Community Foundation

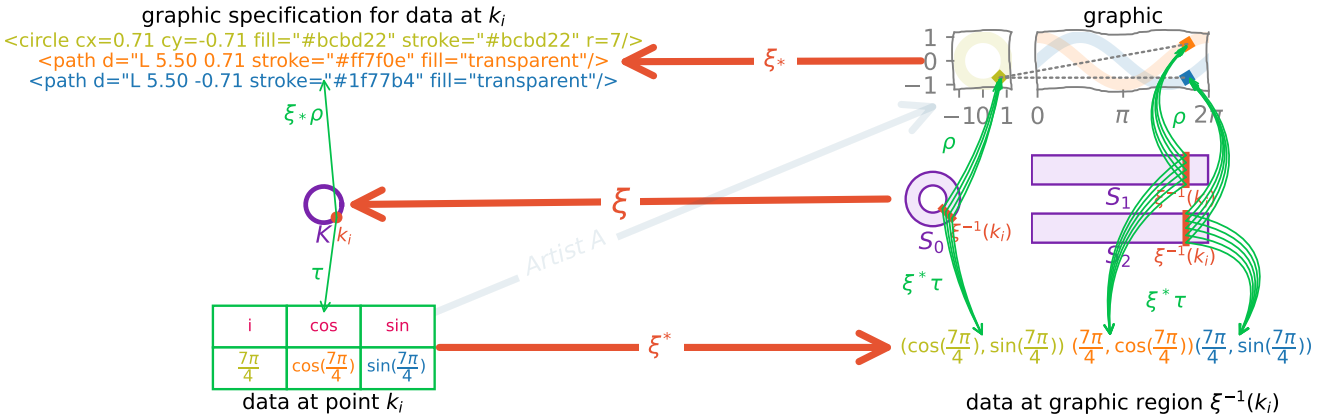


Figure 5: The data τ consists of the sin and cos functions over a unit circle base space K visualized as the circle and two line plots generated from the graphic specification ρ. The index map ξ keeps track of which part of the circle, sin, and cosine plots correspond to which point on the unit circle. The pushforward ξ_* matches each point in the data space to the corresponding graphic specification, and the pullback ξ^* matches each point in the graphic space to the data at that point.

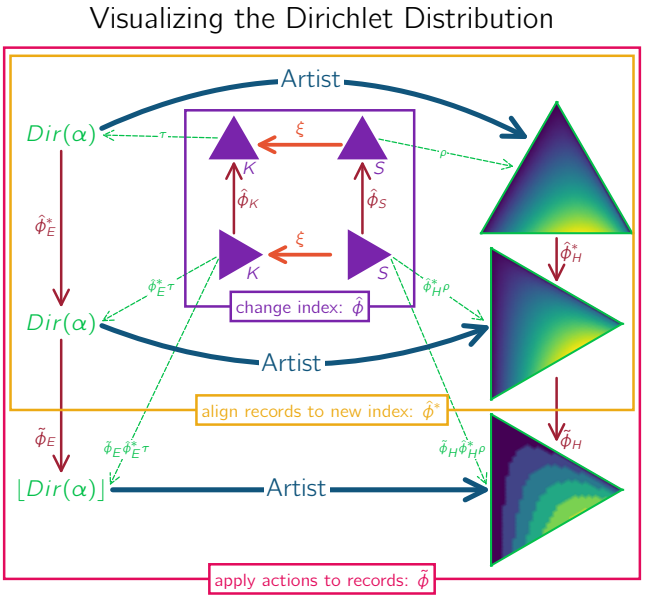


Figure 6: This artist maps a numerical section of the Dirichlet distribution to a corresponding graphical representation. The artist preserves connectivity by not breaking presheaf implementations of rotation $(\hat{\phi}, \hat{\phi}^*)$ of the indexing space K, S and field structure by implementing the data to color mapping such that binning $(\tilde{\phi})$ the data and mapping the bins to colors is equivalent to binning the colors directly. The subscripts denote the space the action occurs in.

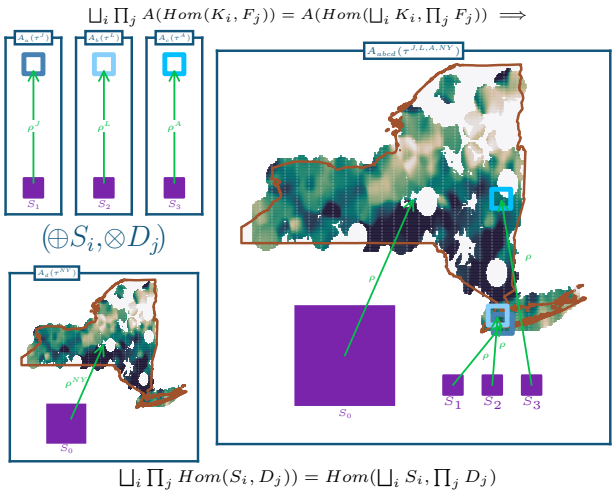


Figure 7: The scatter plot of stations on the rain map from Figure 1 can either be generated as a composition of scatter and map drawing functions or a singular scatter and map drawing function.

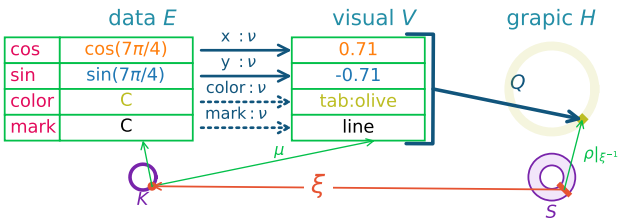


Figure 8: The artist function that draws the green point on the circle is composed of two stages $A = \nu \circ Q$. The encoder ν maps the data values to visual values and the compositor Q generates the graphic specification for producing the sections of the circle. Default encodings, such as color and mark type, are here treated as pullbacks of constant values in the data.

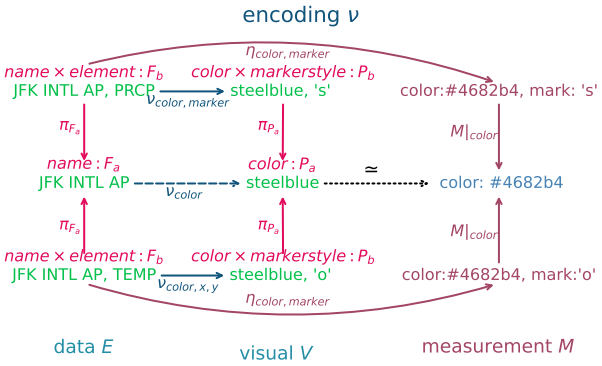


Figure 9: The encoders $\nu_{\mathtt{color,marker}}$ and $\nu_{\mathtt{color}}$ correctly encode JFK INTL AP as steelblue if they return steelblue and it matches the reference color #4682b4. An external testing library can follow a similar procedure using η functions to test library encoders ν.

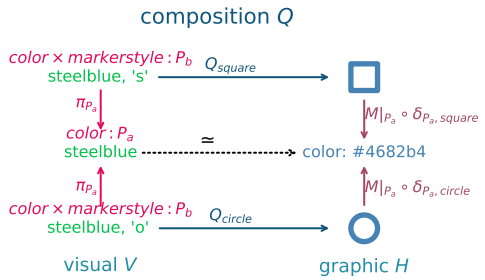


Figure 10: The compositors $\mathrm{Q}_{\mathtt{square}}$ amd $\mathrm{Q}_{\mathtt{circle}}$ correctly compose the visual parameter "steelblue" in their outputs if the measured color is the same in the output of both compositors is equal to the reference color #4682b4.